Vocabulary

Containers, state, and two things called a harness

Robin Langer Claude (Opus 5)

19 September 2026

A working glossary for the framework of Neil Ghani's *Containers for Typed Agentic AI*, fixing four terms that are used loosely across the implementations in `~/git/gold`. The definitions are *stipulative*: they say what each word means here, so that a sentence using it can be checked. The last of them — *harness* — turns out to name two different things, and the note ends by asking which one the harness running a coding agent actually is.

Notation. A container is a pair (P, R), written $P \triangleright R$: a type P of **prompts** (shapes) and, for each $p : P$, a type $R\,p$ of valid **responses** (positions). Equivalently $\sum_{p:P} y^{R\,p}$.

1 The four terms

1.1 Stateful container

A container whose positions are states:

$$\Delta S \;=\; S \triangleright (\lambda_.\, S) \;=\; \sum_{s:S} y^S.$$

Both legs are S: the prompt is the state you are in, the response is the state you are in. It carries no question and no answer — only a place to keep something. Its extension $[\![\Delta S]\!]X = S \times X^S$ is the store comonad.

That the *shapes* are also S is forced rather than incidental: a state you cannot read is not a state, so the prompt type must expose S too. The name records the interesting half.

1.2 State handler

A container morphism whose domain is a tensor of a container and a stateful container,

$$\Delta S \otimes X \;\longrightarrow\; Y.$$

Under the Dirichlet tensor (shapes multiply, positions multiply),

$$\Delta S \otimes X \;=\; (S \times X_0) \triangleright (\lambda(s, x_0).\, S \times X_1 x_0),$$

so such a morphism is exactly a pair

$$\mathrm{fwd} : S \times X_0 \longrightarrow Y_0, \qquad \mathrm{bwd} : \textstyle\prod_{(s,x_0)} (Y_1(\mathrm{fwd}(s, x_0)) \longrightarrow S \times X_1 x_0).$$

The backward leg **splits**, by the universal property of the product in the position, into a state *writeback* component (S) and an ordinary *input back-map* (X_1). That split is the whole reason the tensor is the right home for state. Specialising $X = y$ and $Y = A \triangleright (\lambda_.\, B)$ gives precisely a lens: $\mathrm{get} : S \to A$, $\mathrm{put} : S \times B \to S$.

Two warnings, since "tensor" is load-bearing above.

$\otimes$ *is not* $\triangleleft$. We have $\Delta S \otimes \Delta T = \Delta(S \times T)$ — states multiply, $4y^4$ — whereas $\Delta S \triangleleft \Delta T$ is store-of-store, $8y^4$. The tensor grading is a retract of the composition grading, not a fibre, and the clean writeback split is specifically a tensor phenomenon. A "state handler" composed rather than tensored is a different object and does not have it.

$\otimes$ *is not* $\times$. The categorical product offers both sides and answers *one*; the tensor offers both and answers *both*. In Kelvingrove these are `pick`/$\times$ and `parallel`/$\otimes$ respectively, and swapping them is the easiest mistake in that codebase. "Tensor" above means answer-both.

1.3 Handler; open agent

Both mean: a container morphism. A morphism $(P, R) \to (Q, T)$ is a pair

$$u : P \longrightarrow Q, \qquad f : (p : P) \longrightarrow T(u\,p) \longrightarrow R\,p,$$

control flowing down through u and data flowing back up through f.

They are two names for one thing, taken from two directions. *Handler* names the obligation — something arrives and must be answered. *Open agent* names the means — it is answered by delegating onward, and the openness is the domain it delegates into. A project manager delegating to a developer, an interpreter, a device driver, and an LLM agent calling a tool are all this, and the framework's claim is that they are the same construction rather than four analogies.

Not to be confused with an async function, which is a *position* in a container — a member of some $R\,p$, not a morphism.

The confusion is understandable, because `async f : A -> Promise<B>` is written with an arrow and so looks like the u leg of something. The discriminating test is *legs*: a morphism is a pair of maps, and the second one — the backward leg — is what carries the answer home. An async function has no backward leg, because it is not standing between two interfaces; it is the value that comes back along somebody else's. Awaiting it is consuming a position.

1.4 Harness

In Ghani's sense: a stateful container. The thing that holds S while the work goes on around it.

The Claude Code harness: the subject of the next section, because "is it a container or a container morphism?" turns out to be three questions wearing one coat.

2 Is the Claude Code harness a container or a container morphism?

Asking whether it is a container *or* a morphism is close to asking whether a group is a set or a homomorphism. Container and morphism are object and arrow; a thing is not one to the exclusion of the other. The honest answer is that the harness stands in three relations at once, to three different questions.

1. What does it look like to the model? A container.

Its prompts are the tool calls — `Bash`, `Read`, `Edit`, `Agent`, `ToolSearch` — and for each, the responses are what comes back: an exit code and a stream of bytes, a file's contents, a subagent's report. The dependence is real and checkable: the response type varies with the *value* of the call, not merely its type.

It is a container in the strong sense rather than the loose one. The prompt set is not fixed: *deferred* tools have visible names and no loaded schema, and there is a prompt — `ToolSearch` — whose response **enlarges the set of prompts available afterwards**. A model in which P is a fixed set up front cannot say that; $\triangleleft$ says it immediately, because composition is exactly the operation under which one call's position determines the next call's available shapes.

2. What does it do? A morphism.

It answers the model's prompts by issuing prompts on other interfaces — the operating system, the filesystem, the network, a subagent — and translating the replies back into the response it owed. Control down, data up. That is the definition, with nothing left over.

3. What does it carry? A ΔS.

The working directory, the permission mode, which tool schemas are loaded rather than deferred, the table of background tasks. None of this is in the model's context, and the tell is that the model must be *told* when it changes:

> `Shell cwd was reset to ~/git/gold/from-neil`

Being told is the signature of state one does not own. So the state is real, and it is harness-side.

The resolution

It is already written down, about federation members, and it applies verbatim:

> *each member **has** a container and **is** a morphism: the container is its interface, the morphism is its behaviour.*

The Claude Code harness *has* a container (what the model sees), *is* a morphism (what it does to the world), and *carries* a ΔS (what it remembers between calls). Put the three together and the vocabulary above already has the word for it: a **state handler**, $\Delta S \otimes X \to Y$, with X the interfaces it delegates into and ΔS the session state it keeps.

So the two senses of "harness" do not clash — they compose. The Claude Code harness *is* a harness in Ghani's sense, in the precise sense that it is the thing holding the ΔS that the model does not hold. The equivocation dissolves once one notices that the model and the harness were never candidates for the same slot.

Mai nafka minah. If the harness were merely a container, the question "what happens to the state between two tool calls?" would have no home in the formalism, and the answer would be supplied silently by the implementation. As a state handler it has a home — the writeback component of bwd — which is a thing one can be right or wrong about. That is the practical difference, and it is why the distinction is worth the words.

3 The same four terms elsewhere

here	the note	Kelvingrove (Julia)	stalin / tower (TypeScript)
container	container (P, R)	`Itf`, `AbstractItf`	the container type in `lib/algebra.ts`
handler / open agent	event handler	`Lens` (`fwd`/`bwd`)	morphism in `morphisms.ts`
stateful container ΔS	—	`Itf(S,S)`	—
state handler	—	`Wrkr` — "stateful morphism; state as `Itf(S,S)` $\otimes$ `D`"	`state.ts` (unverified)
tensor $\otimes$	$\boxtimes$	`ParallelItf`, `parallel`	$\boxtimes$ in `lib/algebra.ts`
product $\times$	$\times$	`PickItf`, `pick`	$\times$ in `lib/algebra.ts`

ΔS and the state handler are *not* in the note. They come from MacBeth's third document (15 September 2026), and Kelvingrove's `Wrkr` is the same construction arrived at independently and given a type. The row marked *unverified* is exactly that: `state.ts` has not been read against this definition.

Sources

- Neil Ghani, *Containers for Typed Agentic AI* — `programming-with-containers.pdf`
- MacBeth, Doc #3 — `workers-state-via-tensor.pdf`, and its covering mail
- *My Harness as a Worker* — `my-harness-as-a-worker.pdf`, §1–§2
- *Three container formalisms* — `../kg-notes/container-formalisms.pdf`
- `../kalvingrove/CLAUDE.md`, the vocabulary map and the $\times$/$\otimes$ trap
- `../federation/README.md`, "each member has a container and is a morphism"